

ORQUESTADOR DE AGENTES AUTOHOSPEDADO

SelfHostedContextCrafter

Planificación acotada y auditoría de código
sobre infraestructura local

Documento conceptual — Propuesta de idea

25 de junio de 2026

Un sistema que separa *pensar* (planificar con contexto verificable),
construir (hacer con un humano al control) y *verificar* (auditar contra la fuente),
todo ejecutándose en tu propio hardware.

Resumen ejecutivo

SelfHostedContextCrafter es un orquestador de agentes LLM **autohospedado** diseñado para equipos que desarrollan software bajo restricciones del mundo real — hardware específico, requisitos de negocio y normativas — que no pueden (o no quieren) enviar esa información a un proveedor de IA en la nube.

La idea central es simple pero poco común entre las herramientas actuales: **el agente nunca escribe código de producción**. Su única tarea es leer y digerir toda la documentación técnica relevante (hojas de datos de hardware, requisitos, normativas, contexto de arquitectura) y producir un *plan avanzado* acotado y trazable. Ese plan es lo que un humano — apoyado por su propio agente de código — usa para construir. Una vez que el código existe, un segundo modo del mismo sistema lo **audita** contra el plan y la documentación original, devolviendo una puntuación por módulo, cambios sugeridos y advertencias específicas.

Todo el ciclo se ejecuta en hardware local, con modelos LLM autohospedados, lo que hace viable este enfoque en contextos donde la confidencialidad del hardware, código o normativas es un requisito, no una preferencia.

1 El problema

Los asistentes de código actuales son potentes generando texto, pero comparten tres debilidades recurrentes en proyectos con restricciones del mundo real:

- **Pierden el contexto original.** Generan código con apariencia razonable en el momento, pero sin trazabilidad clara hacia el requisito, normativa u hoja de datos de hardware que lo originó.
- **Alucinan cuando el contexto está disperso.** Cuanto más fragmentada está la documentación (PDFs del fabricante, normativas, especificaciones internas), más fácil es que el modelo rellene vacíos con suposiciones.
- **Requieren enviar información sensible a servicios externos.** El hardware propietario, las normativas internas o el código fuente no siempre pueden salir de la organización.

A esto se suma un problema de **control**: cuando un agente genera y audita su propio código en el mismo paso, no hay verificación independiente real — es el mismo razonamiento revisándose a sí mismo.

2 La idea central

SelfHostedContextCrafter divide el ciclo de desarrollo en tres roles distintos, cada uno con una responsabilidad acotada:

1. Planificar (Modo Planner)

Ingiere toda la documentación técnica disponible y la convierte en un plan de proyecto avanzado. No escribe una sola línea de código de producción: su entregable es la estructura del proyecto y un plan acotado a lo que la documentación realmente dice.

2. Construir (Humano + agente de código)

Un desarrollador, apoyado por su agente de código de preferencia, ejecuta el plan. Aquí es donde se escribe realmente el software — con criterio humano, y con el plan acotado como

guía, no como camisa de fuerza.

3. Auditar (Modo Auditor)

El mismo sistema, conservando la memoria de todo lo ingerido durante la fase de planificación, revisa el código resultante contra el plan original y la documentación fuente. Devuelve una puntuación por módulo, cambios sugeridos y advertencias específicas.

La clave no es ninguno de los tres pasos por sí solo, sino que los tres comparten la **misma base de conocimiento retenida** (RAG). El auditor no audita en el vacío: audita contra exactamente el mismo contexto normativo y técnico que generó el plan, lo que hace que sus advertencias sean trazables hasta la fuente original.

3 Arquitectura de flujo conceptual

3.1 Leyendo el diagrama

- **El Modo 1** normaliza cuatro fuentes de contexto distintas (hardware, requisitos, normativas, contexto específico) a través de un único conversor PDF → MD, para que el Framework trabaje siempre sobre texto homogéneo. La salida no es código: es una estructura de proyecto y un plan avanzado, y el Framework retiene el material fuente como una base de conocimiento persistente.
- **La etapa de construcción** es deliberadamente humana. El plan avanzado y el contexto retenido alimentan al desarrollador y su agente de código de preferencia, pero las decisiones de implementación permanecen en manos humanas.
- **El Modo 2** recibe el código resultante (a través de un conversor dedicado orientado al análisis de código) y el mismo plan avanzado. El Framework en este modo no parte de cero: retiene los RAGs y agentes del Modo 1, por lo que puede auditar el código contra la documentación original, no solo contra el plan.

4 Principios de diseño

Autohospedado por defecto

Modelos LLM ejecutándose en hardware local. Ninguna hoja de datos de hardware, normativa interna o línea de código necesita salir de tu propia infraestructura para que el sistema funcione.

El agente planifica, el humano construye

Separar la generación del plan de la generación de código reduce la superficie de alucinación donde más duele: las decisiones arquitectónicas. El código lo sigue escribiendo quien tiene el criterio y la responsabilidad final.

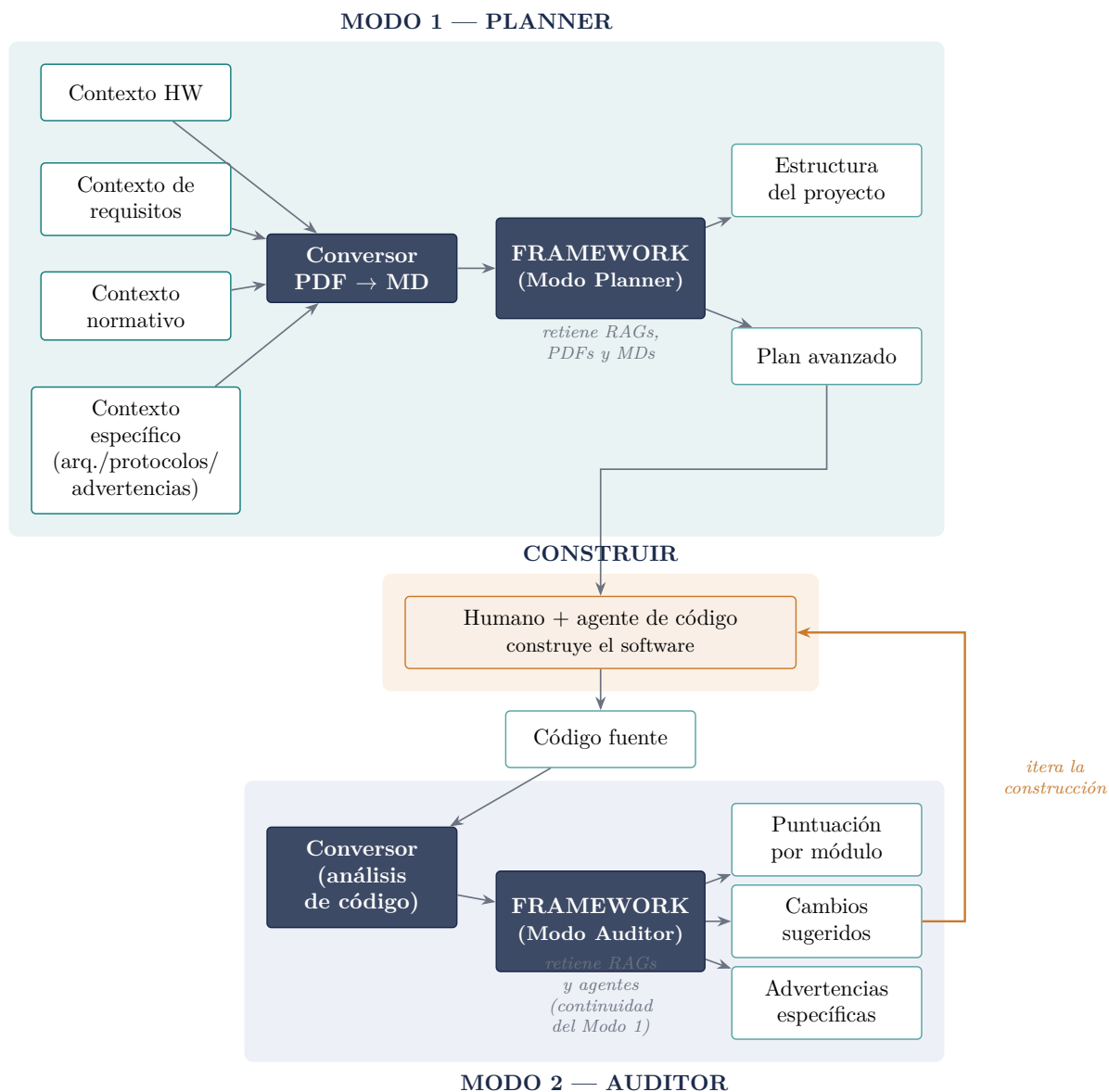


Figura 1: Flujo conceptual: el contexto fluye desde el Planner hacia la etapa de Construcción y de allí al Auditor, sin que el agente toque código de producción directamente. Los cambios sugeridos y las advertencias se realimentan al ciclo de construcción.

Planificación acotada, no plantillas genéricas

El plan avanzado no proviene del conocimiento general del modelo: proviene de lo que la documentación ingerida realmente dice. Eso lo hace más acotado, pero también más fiable y más fácil de justificar en una auditoría real.

Auditoría con memoria, no auditoría ciega

Al retener RAGs y agentes entre modos, el auditor evalúa el código contra la misma fuente de verdad que generó el plan — no contra una interpretación nueva y potencialmente diferente del mismo problema.

5 Dónde encaja esta idea

- Equipos de **firmware o sistemas embebidos** que trabajan con hojas de datos de fabricantes y normativas de seguridad (industrial, automotriz, médica) y que no pueden subir esos documentos a un LLM en la nube.
- Organizaciones con **requisitos de soberanía de datos** que ya tienen, o están dispuestas a invertir en, cómputo local para inferencia de modelos.
- Proyectos donde el **costo de una mala interpretación normativa** es alto, y donde tener un plan trazable a la documentación fuente tiene valor por sí mismo, más allá de la velocidad de desarrollo.

6 En una frase

*“Un orquestador que separa pensar, construir y verificar
— uno que nunca asume lo que la documentación no dice,
y nunca abandona el hardware sobre el que se ejecuta.”*